

• Base line → de operacion (contienen código como IC)
 ↳ Especificacion (sin código) → documentación, definiciones de trabajo
 → puede estar formada por un solo IC

8'50 → importante: Para qué hacemos gestión de configuración? Para mantener la integridad del producto, lo que implica poder responder el qué pasó con cada IC a lo largo del tiempo

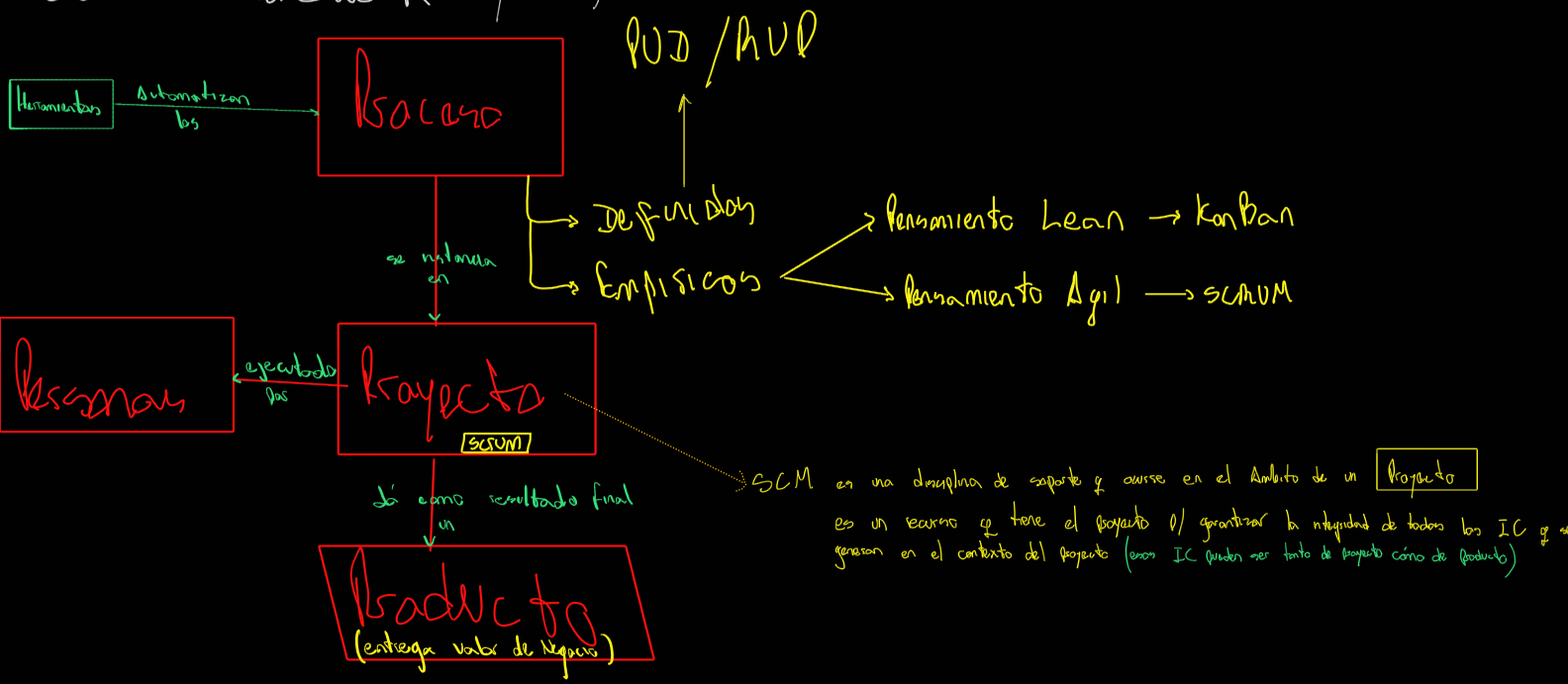
Es a las líneas base a las q se le hace estricto control de cambio → CCC ← interviene cuando se desean cambiar la Línea Base

Auditorias
 (ISO) se planifican en el Plan SCM (con momentos, No con fecha y hora)
 ↳ se hacen en un orden
 ↳ se hacen a una base line en particular, no a todo el tiempo
 ↳ revisión objetiva e independiente
 ↳ la hace alguien de afuera del proyecto
 ↳ sin dependencia con el jefe del proyecto
 ↳ 1º la Física
 ↳ 2º la Funcional

→ Física: Se Audita si se cumple con lo dicho en el Plan SCM: Reglas de Nomenclatura, Ubicaciónes, etc.
 → Funcional: Apunta a
 ↳ Herramientas: Check list

14'50 (importante)

SCM ⇒ Framework y Agilismo es una filosofía / pensamiento



SCM es una disciplina de soporte q surge en el ámbito de un Proyecto
 es un recurso q tiene el proyecto o/ garantizar la integridad de todos los IC q se generan en el contexto del proyecto (esos IC pueden ser tanto de proyecto como de producto)

Pilares Empíricos: • Inspección (puntos de control)
 • Adaptación
 } ciclos cortos de inspección - Adaptación

Controlar q el team
vaya ganando experiencia
en cada ciclo corto

• Transparencia (No fingir demencia, no mentir, no guardar cosas info)
Respecto a los ciclos cortos funcionan. Es la base d q la información funciona y la adaptación se puede realizar

Problemas No visibles → No se pueden solucionar → No hay Aprendizaje

Definidos dicen "La experiencia si es extrapolable"
→ Describen el proceso lo más detallado posible describiendo todo lo necesario al team software (POs)
→ flexibilidad para alcanzar la predictibilidad
- No se impone determinista
- team q el proceso se puede usar aunque de la misma forma → se tienen que adaptar / ajustar en mayor medida
- ajustar es su naturaleza

Manifiesto Agil → 12 Principios
→ 4 Valores
→ Colaboración con el cliente sobre Negociaciones Contractuales

Ningún Rol tecnico puede asumir el rol de PO

Empiricos → vision: - lo más importante para hacer la soft es la gente
- Cada proyecto es una instancia única e irrepetible de experiencia que no se la puede llevar a otro lado (?)
→ Filosofía Agil adhace?

una Story → es la tecnica
¿Por qué en Agiles los requerimientos Agiles?

Salida ≠ Resultado 1º 23'
output out come

contenedor de los hq's Agiles = Product Backlog 1º 29'

Lider de Proyecto no existe en Agilismo

Scrum Master → tecnico

de Scrum no dice nada de cómo hacer el producto. Hq. como personas capacitadas q saben hacer su trabajo (46:30 escuchan está. Hacen la 50:30)
y si mi cliente no es Agil, me conviene trabajar con un proceso definido

Min 561

Ingeniería y Calidad de Software

Ciclo Lectivo 2026 – Segundo Cuatrimestre



UTN-FRC

INGENIERÍA Y CALIDAD DE SOFTWARE



Filosofía Ágil

Manifiesto Ágil

Procesos Empíricos

→ Basados en experiencia → No se entragola
¿cómo gana experiencia?
↓
HACIENDO



VALORES ÁGILES

INDIVIDUOS E
INTERACCIONES



SOBRE



PROCESOS Y
HERRAMIENTAS

SOFTWARE
FUNCIONANDO



SOBRE



DOCUMENTACION
EXTENSIVA

COLABORAR CON
EL CLIENTE



SOBRE



NEGOCIACION
CONTRACTUAL

RESPONDER
AL CAMBIO



SOBRE



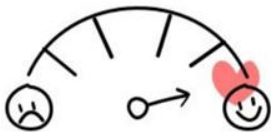
SEGUIR UN PLAN



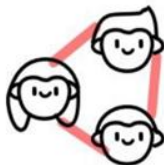
Los 12 principios del Manifiesto Ágil

Técnicos y No Técnicos Trabajando juntos
Todo el Proyecto

1 Nuestra mayor prioridad es **satisfacer al cliente.**



4 Los responsables de negocios, diseñadores y desarrolladores deben **trabajar juntos** día a día durante el proyecto.



2 **Aceptar** que los requisitos cambien.



5 Desarrollamos proyectos en torno a **individuos motivados.**



3 **Entregar software funcional** frecuentemente.



6 El método más eficiente de comunicar información es **conversaciones cara a cara.**



→ Complicado debido a la Ev. de Tecnología

7 El **software funcionando** es la principal **medida de éxito.**



8 Los procesos ágiles promueven el **desarrollo sostenible.**



10 La simplicidad es esencial.



11 Las mejores arquitecturas, requisitos, y diseños **emergen** de **equipos auto-organizados.**

"No lo So"
Explicarlo comparándolo con los procesos definidos
Los diseños se tienen justificados y se logran todo en los dev (Ej. estimaciones y comparaciones)



se comparan a tal fin y después cumplir con la entrega

9 La **atención continua a la excelencia técnica** y al buen **diseño** mejor la Agilidad.



12 A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo y de acuerdo a esto **ajustan su comportamiento.**



¿Qué es Ágil?

→ Por el problema de hacer software es un problema social
Actividad Humana Intensiva

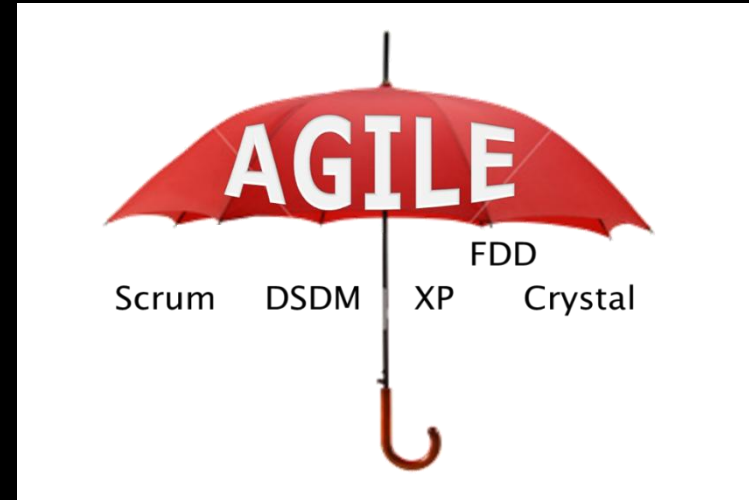
NO es una metodología o proceso

Ágil es una **ideología** con un conjunto definido de principios que guían el desarrollo del producto

Valores de los equipos ágiles ...

- Planificación continua, multi-nivel
- Facultados, auto-organizados, equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing Concurrente

• El elemento más importante de Ágil es la gente



Desarrollo ágil de Software (Agile)

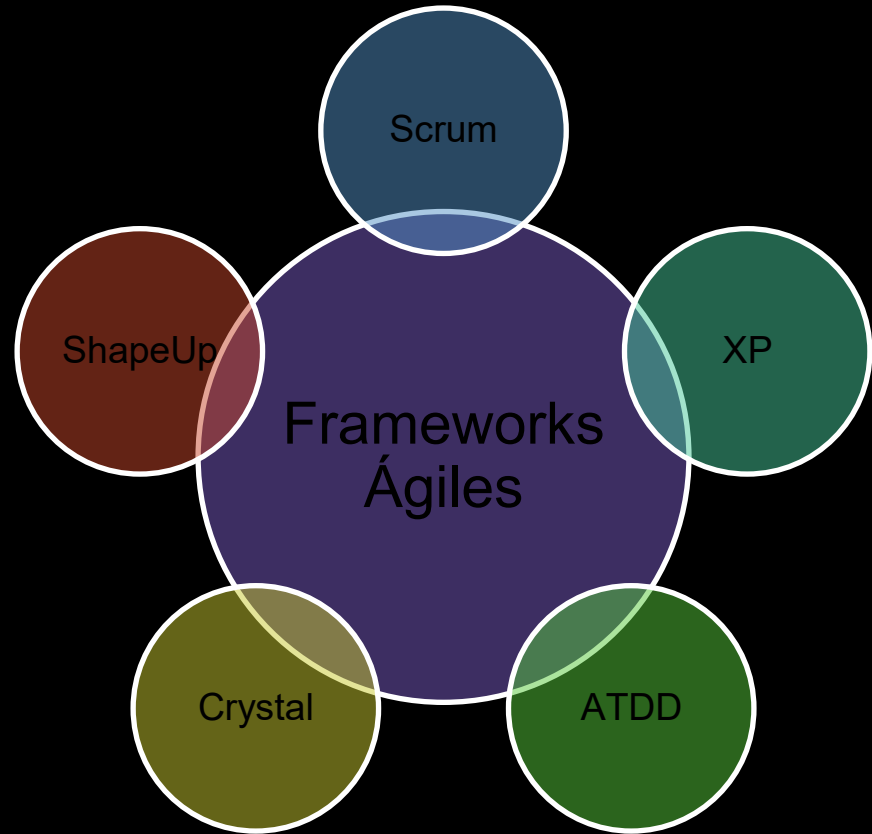
- Un compromiso útil entre nada de proceso y demasiado proceso (Fowler, 2001)

¿Pero qué significa Ágil?

** mantener un equilibrio entre los extremos de los procesos definidos y el caos de el caos code on /*

- **Balance entre ningún proceso y demasiado proceso.** La diferencia inmediata es la exigencia de una menor cantidad de documentación, sin embargo no es eso lo más importante:
 - Los métodos ágiles son adaptables en lugar de predictivos.
 - Los métodos ágiles son orientados a la gente en lugar de orientados al proceso.

Algunos frameworks ágiles



¿Por qué ir a Agile?



87%

Ability to manage changing priorities



85%

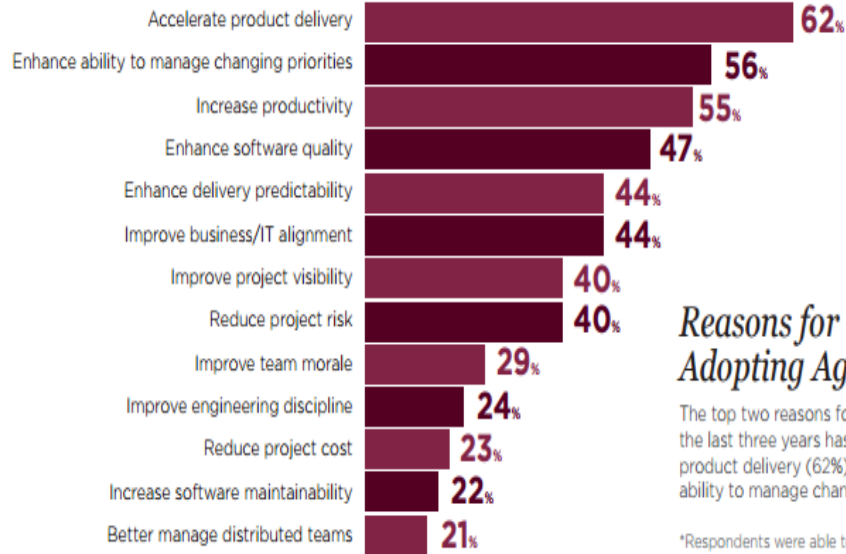
Increased team productivity



84%

Improved project visibility

Top 3 Benefits of Agile

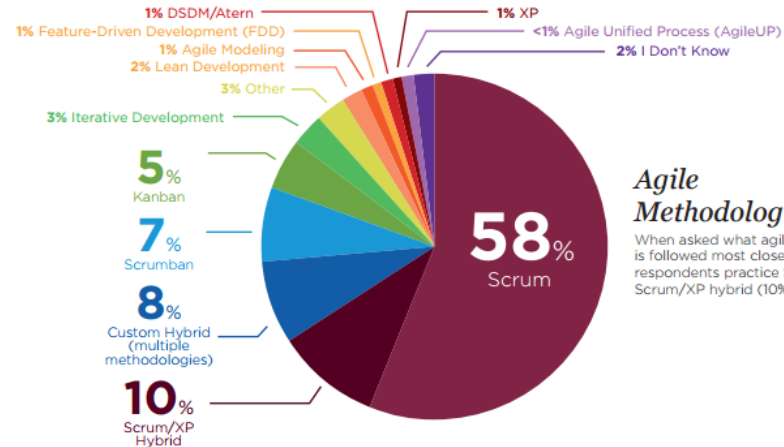


Reasons for Adopting Agile

The top two reasons for adopting the last three years has been to accelerate product delivery (62%) and enhance ability to manage changing priorities (56%).

*Respondents were able to make multiple selections.

AGILE METHODS AND PRACTICES



Agile Methodologies Used

When asked what agile methodology is followed most closely, nearly 70% of respondents practice Scrum (58%) or Scrum/XP hybrid (10%).

Técnicas efectivas

Agile Techniques Employed

More than 39% of the respondents practiced Kanban within their organizations, up from 31% in 2014. Conversely, iteration planning dropped slightly from 71% in 2014 to 69% in 2015, likely indicating a transition to more flow-based methods such as Lean and Kanban.

TOP 5 AGILE TECHNIQUES



83%

DAILY
STANDUP



82%

PRIORITIZED
BACKLOGS



79%

SHORT
ITERATIONS



74%

RETROSPECTIVES

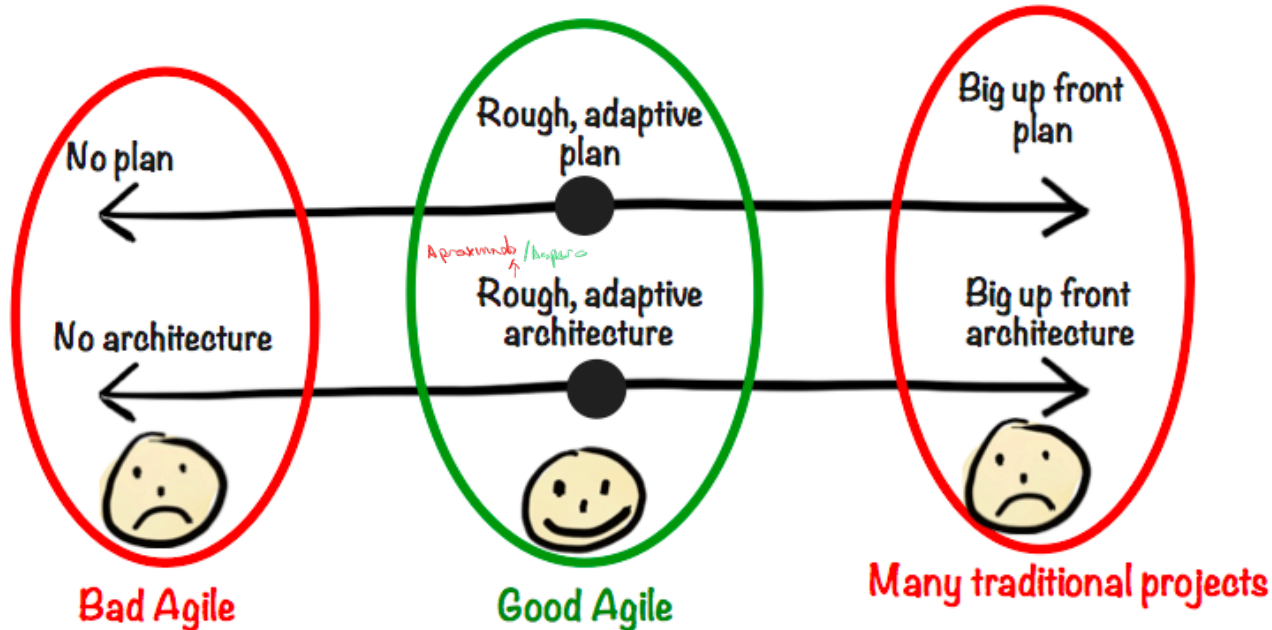


69%

ITERATION
PLANNING

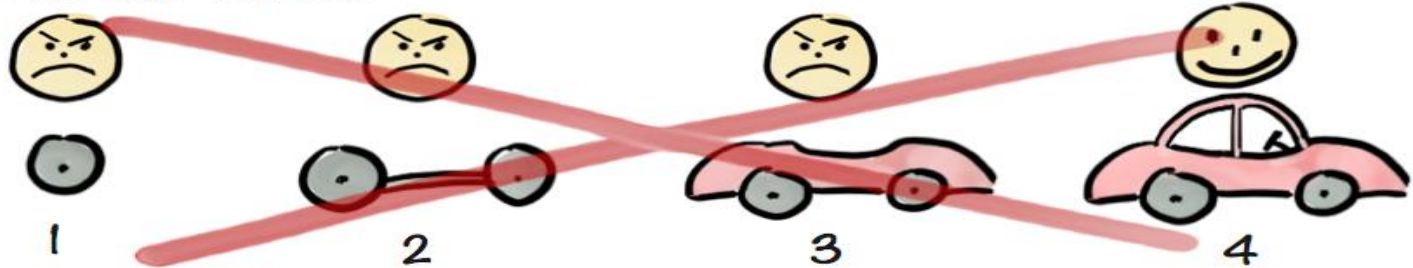
Ser Ágil no es ser indisciplinado.

Don't go overboard with Agile!

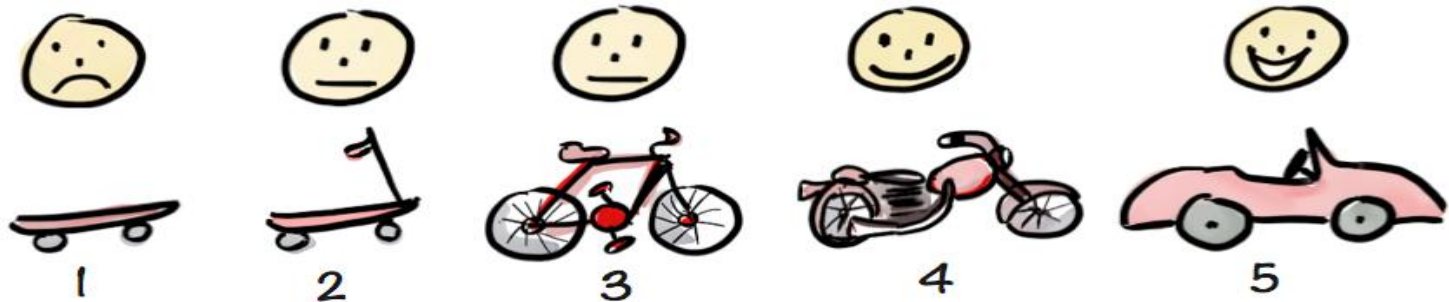


Entonces hacemos todo por pedacitos y somos
agiles!!!?? **NO!**

Not like this.... *Así NO...*

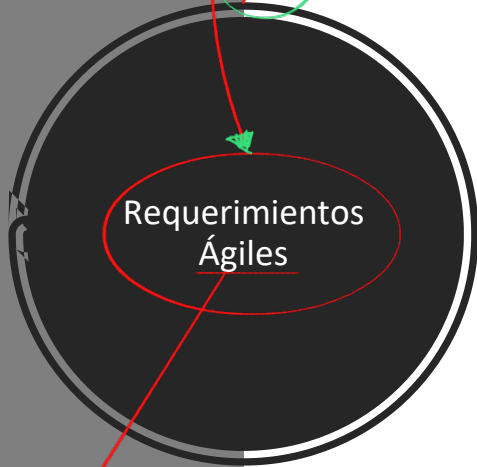


Like this! *Así!*



Técnica de Req's Ágiles = UG

Para identificar
requerimientos



soft funcionando
por sobre
Documentación

satisfacer al cliente con
entregas frecuentes y temporales
de software funcionando

potencialmente desplegado
ciclos de vida iterativos



PROBLEMA / NECESIDAD / OPORTUNIDAD

[PONGA AQUÍ EL
CAMINO MÁS EFECTIVO]



SOLUCIÓN / ENTREGABLE

Requerimientos en Agile

¿Por qué son Ágiles los
Req's Ágiles?

Porque Abrazan el cambio y
no intentan especificar todos
al inicio del proyecto

↓
FINAL

1º Partimos de la visión que tiene el PO
de cómo Quiere el Producto (es distinto a lo que
necesita en realidad)



Usar el "valor"
para construir
el producto
correcto

→ valor de Negocio: No Alcanza sólo con hacer software, sino q debemos
nuestro soft q le sirva al negocio o cumplir con su
objetivo

Resultado (output) ≠ Salida (output)



Usar historias y modelos para mostrar
que construir

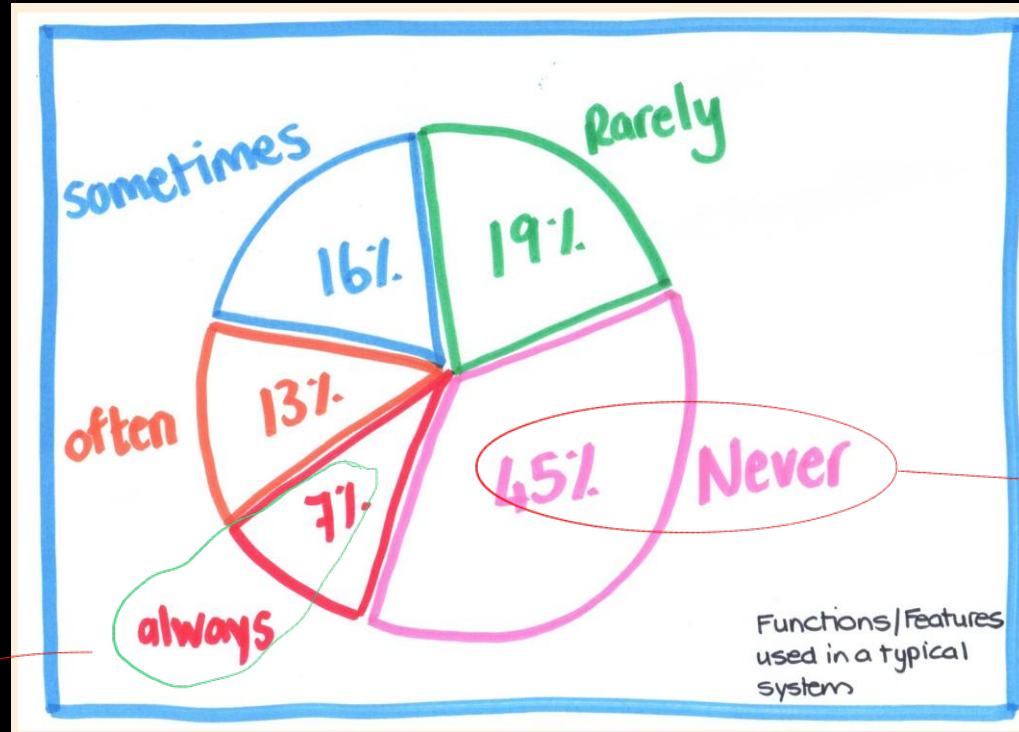
→ Herramientas de comunicación y Almacenamiento
entre el PO y el team para explicar
el caso se quiere construir

Determinar
que es "sólo lo
suficiente"

→ Construir lo mínimo necesario q satisficiera la
necesidad actual eliminando el waste

El costo del tradicional BRUF

Los productos
“Exitosos”
también tienen
un desperdicio
significante

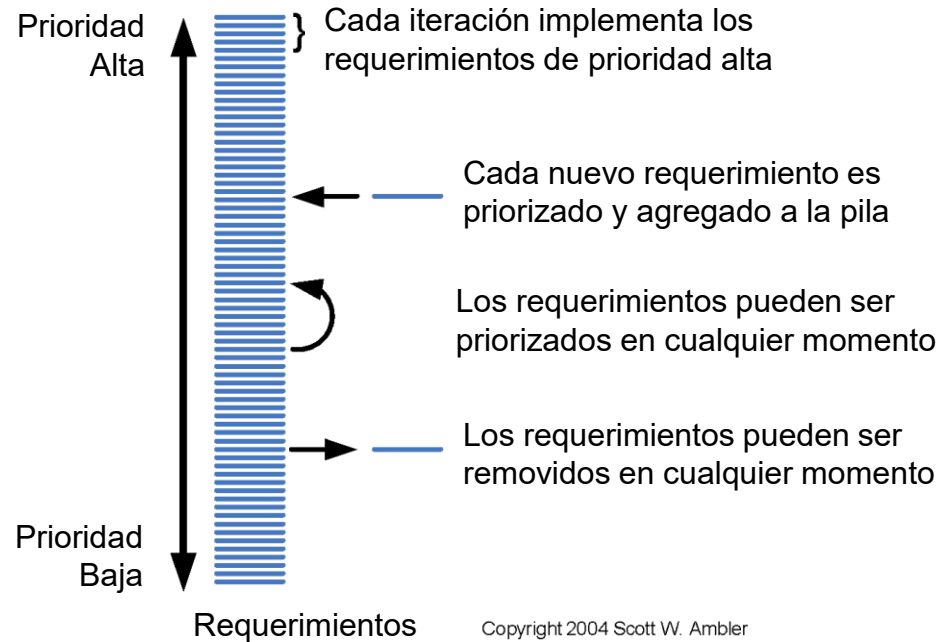


Hay que pagarle a esa
7% (dar en el blanco)

Fuente: Jim Johnson of the Standish Group, Keynote Speech XP 2002

Gestión Ágil de Requerimientos de Software

Los requisitos cambiantes son una ventaja competitiva
si puede actuar sobre ellos



Copyright 2004 Scott W. Ambler

El CORAZON del concepto de Requisitos Ágiles está sustentado en IT

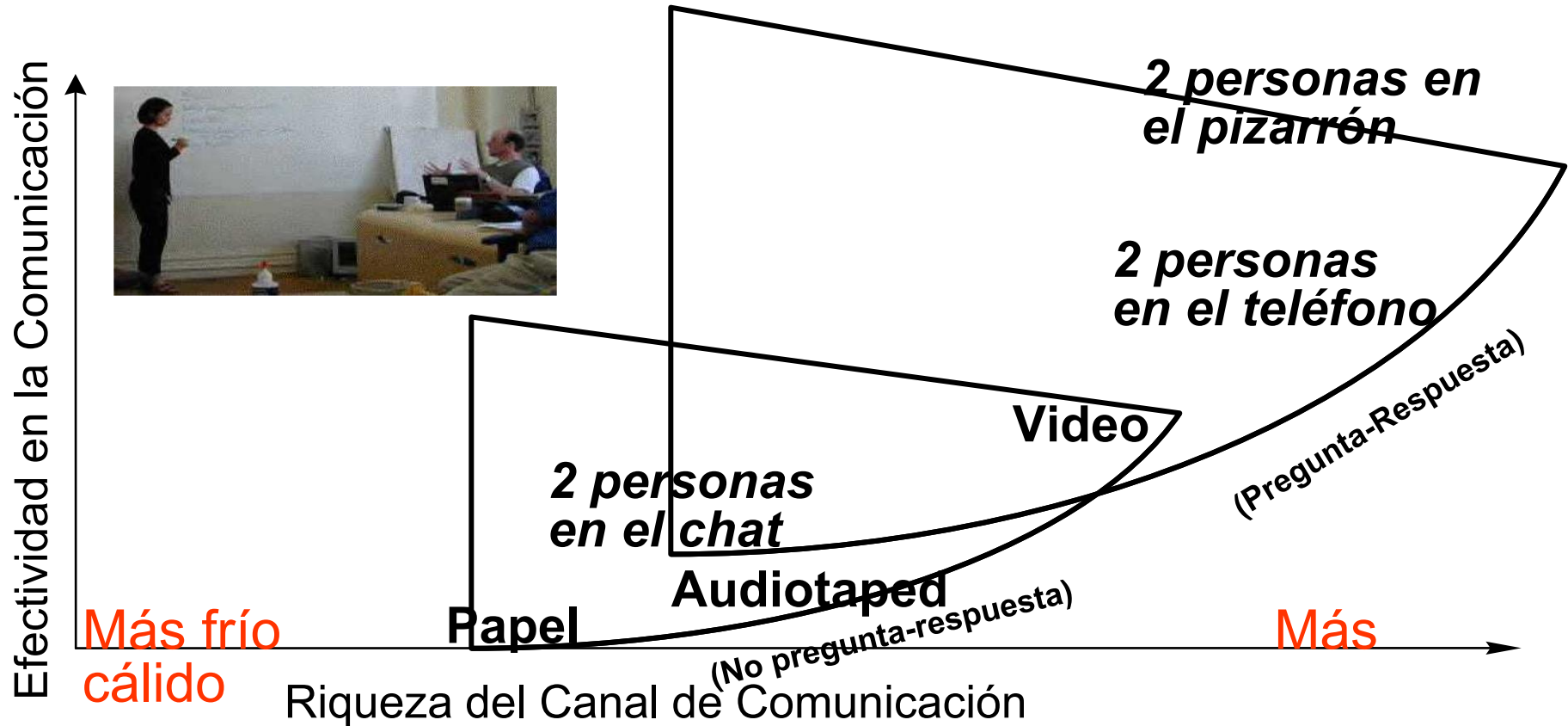


just in time

No invertir tiempo ni recursos antes en el Product Backlog el 100% de los días al principio del proyecto para tener un mejor producto.
Debemos empezar a trabajar con una VISION de lo que queremos hacer con el producto.
La 1ª vez que es la base de los 100 días.

**Analice cuando lo
necesite, no antes**

El cara-a-cara permite que fluya información
vocal, subvocal, gestual con realimentación rápida.



Esto dice que es más bueno que estar copiando hys, algunos se le pasan $\Rightarrow$ requerimientos emergentes

*“**Valor** es la obtención de
**beneficio tangible o
intangible**”*

Masa Maeda, Serious LeAP

*“El **valor** lo asociamos a la
**utilidad, beneficio o
satisfacción** que le ofreces a
los usuarios finales **por cada
funcionalidad completa que le
entregas**”*

Pablo Lischinsky, Agile Trainer &
Consultant, Entrepreneur

Tipos de Requerimientos



Requerimiento de Negocio

Disminuir un X% de tiempo invertido en procesos manuales relacionados con atención al cliente.



Requerimiento de Usuario

Realizar consultas en línea del estado de cuenta de los clientes



Requerimiento Funcional

Generar reporte de saldos de cuenta. Recibir notificaciones por mail



Requerimiento No funcional

Formato del reporte PDF. Cumplir niveles de seguridad para credenciales de usuarios según la ley bancaria 9999XX



Requerimiento de Implementación

Servidores en la nube

Requerimientos de Negocio

Requerimientos de Usuario

Requerimientos de Software



Dominio del Problema

→ US

están en
Algunas se expresan en el ☐ que describe
el mundo que el software tenga

→ CU



Descrito con la interacción del
usuario - sistema

Dominio de la
Solución



Tradicional vs. Ágil

¿Cuál variable permanece fija?

Tradicional: Alcance

Ágil: **Tiempo** y **recursos** (equipo)

FIJO →

ESTIMADO →

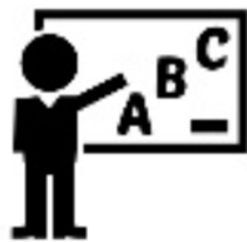


→ es regulable en el tiempo
definido y para ese equipo

EN RESUMEN...



ENTENDIENDO LA
NECESIDAD Y NEGOCIO...



DESCUBRIENDO LA SOLUCIÓN
DE FORMA COLABORATIVA...



JUNTO A UN EQUIPO
MOTIVADO Y COMPETENTE...



ENTREGAMOS
FRECUENTEMENTE VALOR A
LOS STAKEHOLDERS.

Por último



Los cambios son la única constante.



Stakeholders: no son todos los están.



Siempre se cumple eso de que: “El usuario dice lo que quiere cuando recibe lo que pidió”.



No hay técnicas ni herramientas que sirvan para todos los casos.



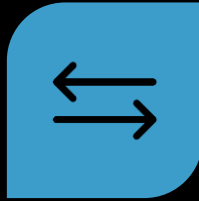
Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de “valor”.

Ves

Principios Ágiles relacionados a los Requerimientos Ágiles



1- LA PRIORIDAD ES SATISFACER AL CLIENTE A TRAVÉS DE RELEASES TEMPRANOS Y FRECUENTES (2 SEMANAS A UN MES)



2 -RECIBIR CAMBIOS DE REQUERIMIENTOS, AUN EN ETAPAS FINALES



4 - TÉCNICOS Y NO TÉCNICOS TRABAJANDO JUNTOS TODO EL PROYECTO



6 - EL MEDIO DE COMUNICACIÓN POR EXCELENCIA ES CARA A CARA



11 - LAS MEJORES ARQUITECTURAS, DISEÑOS Y REQUERIMIENTOS EMERGEN DE EQUIPOS AUTOORGANIZADOS

Principios relacionados a los Requerimientos Ágiles

Principios Ágiles

- 1- La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes (2 semanas a un mes)
- 2 -Recibir cambios de requerimientos, aun en etapas finales
- 4 - Técnicos y no técnicos trabajando juntos TODO el proyecto
- 6 - El medio de comunicación por excelencia es cara a cara
- 11 - Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados

Principios Lean

- 1- Eliminar el desperdicio
- 4 - Diferir Compromisos
- 6 - Ver el todo

User Stories

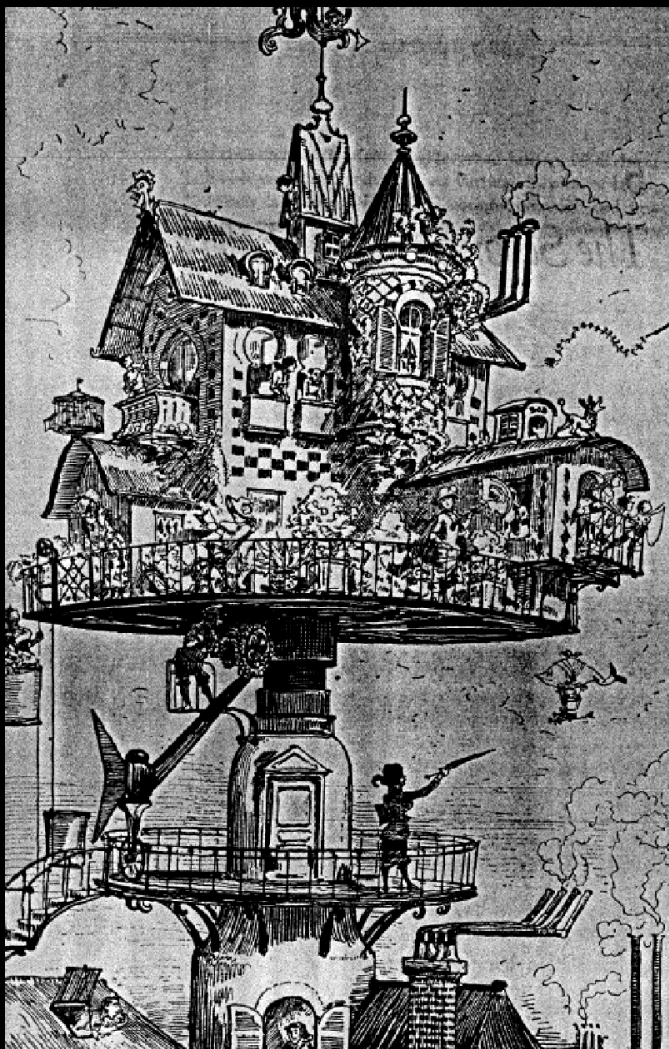
“....se las llama “stories” porque se supone que Ud. cuenta una historia. Lo que se escribe en la tarjeta no es importante, lo que Ud. habla, si!.

--- Jeff Patton, InfoQ,

create conversation.



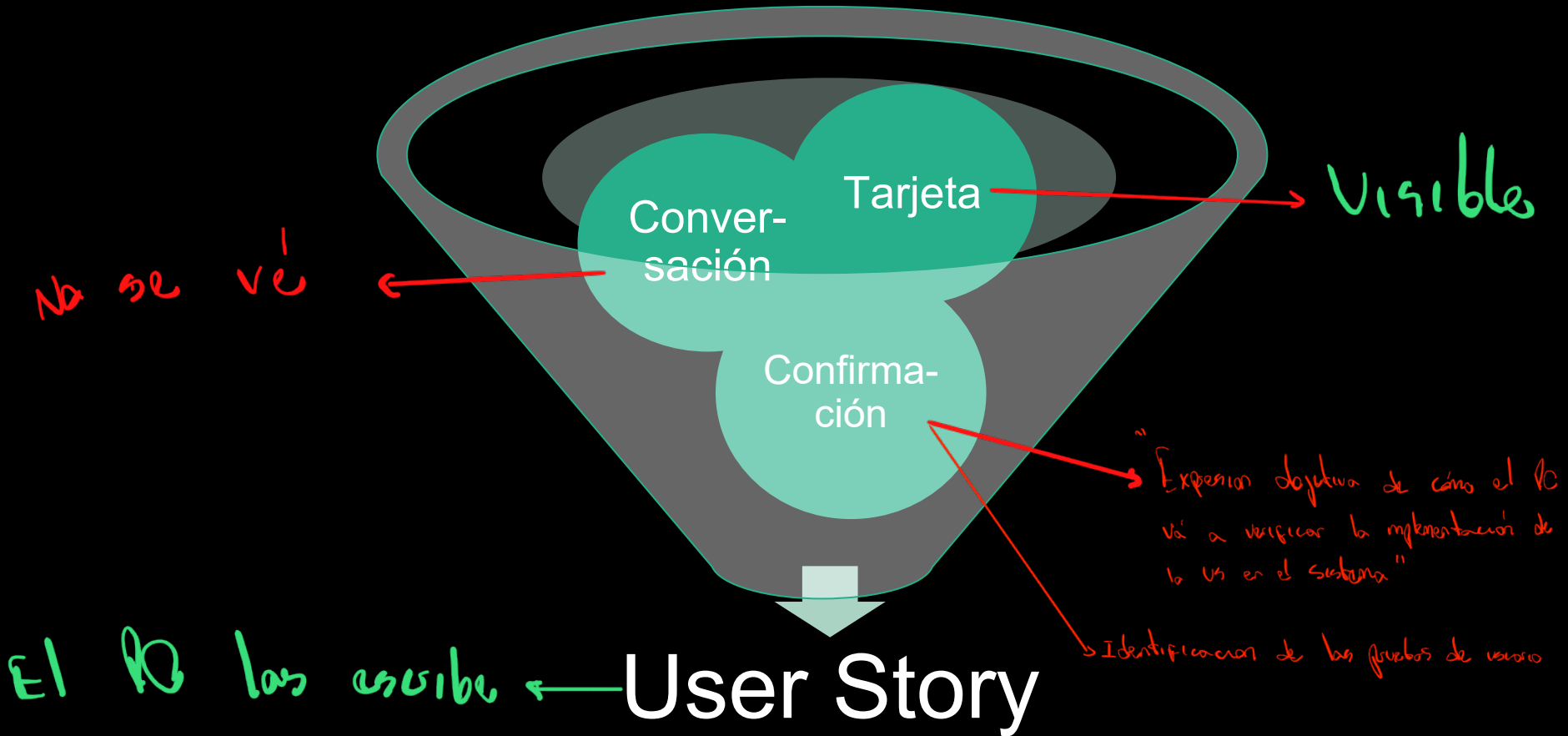
@gapingvoid



La parte más difícil de construir un sistema de software es decidir precisamente qué construir. Ninguna otra parte del trabajo conceptual es tan difícil como establecer los requerimientos técnicos detallados... Ninguna otra parte del trabajo afecta tanto el sistema resultante si se hace incorrectamente. Ninguna otra parte es tan difícil de rectificar más adelante”

Fred Brooks - “No Silver Bullet - Essence and Accidents of Software Engineering”. IEEE Computer, Abril de 1987.

¿Cuáles son las partes de una User Story?



Forma de expresar las Historias de Usuario

As who, I want
what so that why.

Como **<nombre del rol>**,
yo puedo **<actividad>**
de forma tal que **<valor**
de negocio que
recibo>

→ sirve para la priorización de la US

Comunica **porque es**
necesaria la actividad

Representa **quién**
está realizando la
acción o **quién recibe**
el valor de la
actividad.

Representa la
acción que
realizará el
sistema

User Story: un ejemplo de tarjeta

Resumen de la User

Frase verbal

Buscar Destino por Dirección

Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

¿Qué es una User Story?

Front of Card

173

As a student I want to purchase
a parking pass so that I can
drive to school

Priority: ~~High~~ Should
Estimate: 4

Back of Card

Confirmations:

- ~~The student must pay the correct amount~~
- One pass for one month is issued at a time
- The student will not receive a pass if the payment isn't sufficient
- The person buying the pass must be a currently enrolled student.
- The student may only buy one pass per month.

Las User Stories son Multipropósito

- Las historias son:
 - Una necesidad del usuario
 - Una descripción del producto *de funcionalidad*
 - Un ítem de planificación *del Product Backlog*
 - Token para una conversación *?*
 - Mecanismo para diferir una conversación *?*

** Kent Beck coined the term user stories in Extreme Programming Explained 1st Edition, 1999*



El Product Owner Prioriza las historias en el Product Backlog

Es el contenedor de las Píldas Ágiles

- Lista Priorizada de WorkItems
- Contenedor de los items del product Backlog

ordenados por prioridad

US (User Stories para representar estos items Priorizados por la mayoría de frameworks Ágiles)

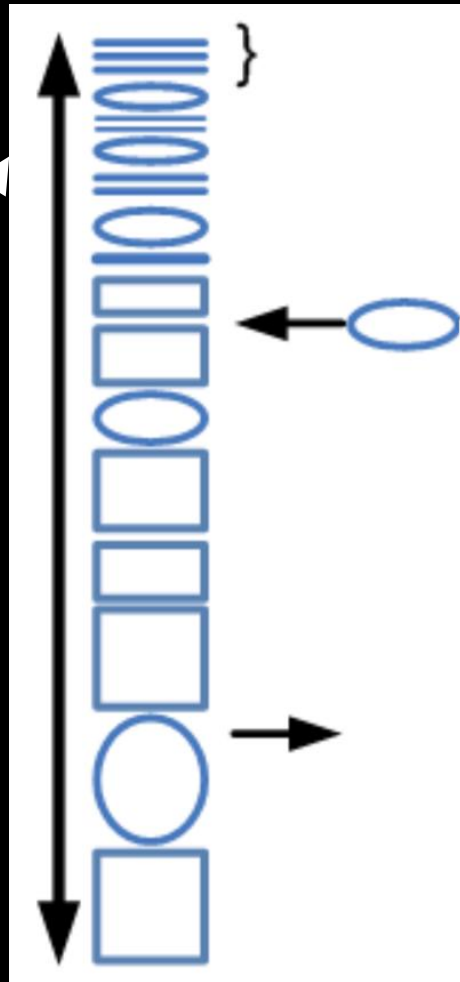
- su dueño es el PO (definición ó última palabra es de él)

Prioridad Alta

Modelado con mayor detalle

Modelado con menor detalle

Prioridad Baja



Ítems de Trabajo

Cada iteración implementa los ítems de trabajo de mayor prioridad

Cada nuevo requerimiento es priorizado y agregado a la pila

Los ítems de trabajos pueden ser re-priorizados en cualquier momento

Los ítems de trabajo pueden ser removidos en cualquier momento

User stories: **Porciones Verticales**



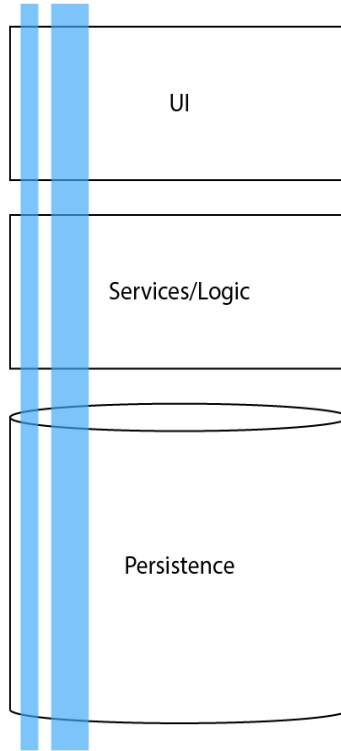
Donde impacta
“fuertemente”
esto??

Story 1	Story 2	
GUI		
Business Logic		
Database		

User stories: Porciones Verticales

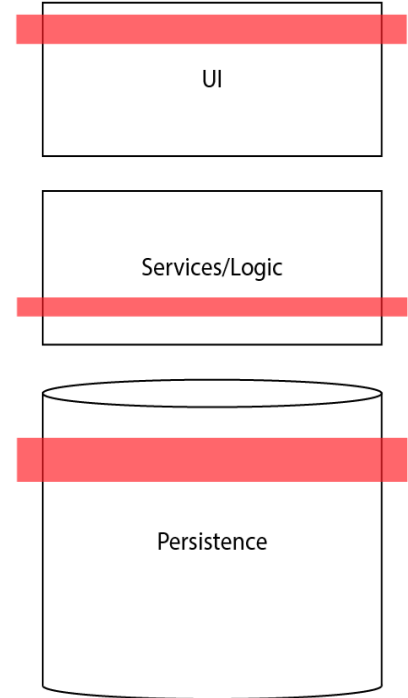
Vertical Slices

include changes to each architectural layer sufficient to deliver an increment of value



Horizontal Slices

multiple slices must be completed to deliver an increment of value



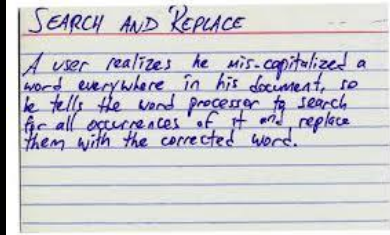
Modelado de Roles



Modelado de Roles: Tarjeta de Rol de Usuario

Rol de usuario: Reclutador Interno

Nos es un experto en computadoras, pero bastante adepto a utilizar la Web. Utilizará el software con poca frecuencia pero muy intensamente. Leerá anuncios de otras compañías para averiguar cuál es la mejor palabra para sus anuncios. La facilidad de uso es importante, pero más importante es que lo que aprenda, lo pueda recordar meses después.



Modelado de Roles: Técnicas Adicionales

- Personas

Mario trabaja como reclutador en el departamento de Speedy Networks, una fábrica de componentes de red de alta gama. El ha trabajado para Speedy Networks por 6 años. Mario tiene un arreglo de horario flexible y trabaja desde casa cada viernes. Mario es muy fuerte con las computadoras y se considera a sí mismo un usuario avanzado de los productos que usa. La esposa de Mario, Kim, está terminando su Doctorado en Química en la Universidad de Stanford. Dado que Speedy Networks ha estado creciendo consistentemente, Mario siempre está buscando ingenieros.



Modelado de Roles: Técnicas Adicionales – Personajes Extremos

Diseño de un PDA para:

- El Papa
- una mujer de 20 años con muchos novios
- un traficante de drogas

Tanto la mujer como el traficante desearán mantener agendas separadas en caso de que la vea la policía o un novio. El Papa probablemente tenga menos necesidad de discreción pero querrá un tamaño de fuente más grande.

Usuarios Representantes (Proxies)

- Tipos de usuarios representantes:
 - Gerentes de Usuarios
 - Gerentes de Desarrollo
 - Alguien del grupo de marketing
 - Vendedores
 - Expertos del Dominio
 - Clientes
 - Capacitadores y personal de soporte.

*No son ideales como
los usuarios verdaderos
...EVITELOS !!!*

Criterios de Aceptación de User Stories



User Story



User Story



Criterios de Aceptación de Historias de Usuario

- Definen límites para una user story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos)
- Ayudan a que el equipo tenga una visión compartida de la US
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad en una US



User Story: un ejemplo de tarjeta

Buscar Destino por Dirección

Como Conductor quiero buscar un destino a partir de una calle y altura para llegar al lugar deseado sin perderme.

Criterios de Aceptación:

- La altura de la calle es un número. → puede faltar q' es opcional
- La búsqueda no puede demorar más de 30 segundos. → hNF

¿Cuáles son los Criterios de Aceptación buenos?

- **Definen una intención**, no una solución
 - Ej.: El usuario debe elegir al menos una cuenta para operar
- **Son independientes de la implementación?**
- **Relativamente de alto nivel**, no es necesario que se escriba cada detalle



¿Y los detalles? ¿Dónde van?



- Detalles como:
 - El encabezado de la columna se nombra “Saldo”
 - El formato del saldo es 999.999.999,99
 - Debería usarse una lista desplegable en lugar de un Check box.
- Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:
 - Documentación interna de los equipos
 - Pruebas de aceptación automatizadas

→ se definen a partir de los criterios de aceptación

Pruebas de Aceptación de User Stories

Front of Card

1B

As a student I want to purchase
a parking pass so that I can
drive to school

Priority: ~~High~~ Should
Estimate: 4

Back of Card

Confirmations:

- ~~The student must pay the correct amount~~
- One pass for one month is issued at a time
- The student will not receive a pass if the payment isn't sufficient
- The person buying the pass must be a currently enrolled student.
- The student may only buy one pass per month.

Pruebas de Aceptación de Historias de Usuario

Expresan detalles resultantes de la conversación

Complementan la User Story

Proceso de dos pasos:

1. Identificarlas al dorso de la US.
2. Diseñar las pruebas completas



User Story: Tarjeta y Pruebas de Aceptación

Buscar Destino por Dirección

Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

Criterios de Aceptación:

- La altura de la calle es un número.
- La búsqueda no puede demorar más de 30 segundos.

Pruebas de Usuario

- ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa).
- ❑ Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla).
- ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla).
- ❑ Probar buscar un destino en un país inexistente (falla).
- ❑ Probar buscar un destino en País existente, ciudad inexistente (falla).
- ❑ **Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).**

Ejemplo: para un software de un GPS

Como Conductor quiero buscar un destino a partir de sus coordenadas para conocer el camino a recorrer para llegar al destino deseado.

Ejemplo:

Como **Conductor** quiero **buscar un destino a partir de sus coordenadas** para **conocer el camino a recorrer** para llegar al destino deseado.

Criterios de Aceptación: Las coordenadas se representan con tres números que indican longitud y tres números que indican latitud. Cada número representa los grados, minutos y segundos respectivamente. Además se debe indicar la orientación (norte, sur, este, oeste).

- Probar buscar un destino en un país y ciudad existentes, de dos coordenadas existentes (pasa).
- Probar buscar un destino en un país y ciudad existentes, de una coordenada inexistente (falla).
- Probar buscar un destino en un país y ciudad existentes, de dos coordenadas existentes sin indicar la orientación (falla).
- Probar ingresar coordenadas de

Ejemplo: User Stories / Casos de Prueba

Como compañía quiero pagar por una búsqueda de puestos con una tarjeta de crédito, así resuelvo mi necesidad en forma más eficiente.

Criterio de Aceptación:

- Se acepta Visa, MasterCard y American Express
- En compras mayores de \$100 se piden el número del dorso de la tarjeta

Probar con Visa (pasa)

Probar con MasterCard (pasa)

Probar con American Express (pasa)

Probar con Dinner's Club (falla)

Probar con números de tarjeta buenos

Probar con números de tarjeta malos

Probar con números de tarjeta faltantes

Probar con tarjetas vencidas

Probar con montos menores de \$100

Probar con montos mayores de \$100

Definición de listo – Definition of Ready



Definición de Hecho – Definition of Done



*INVEST Model

- **Independent** – calendarizables e implementables en cualquier orden
- **Negotiable** – el “qué” no el “cómo”
- **Valuable** – debe tener valor para el cliente
- **Estimatable** – para ayudar al cliente a armar un ranking basado en costos
- **Small** – deben ser “consumidas” en una iteración
- **Testable** – demostrar que fueron implementadas

* “INVEST in Stories” – Bill Wake

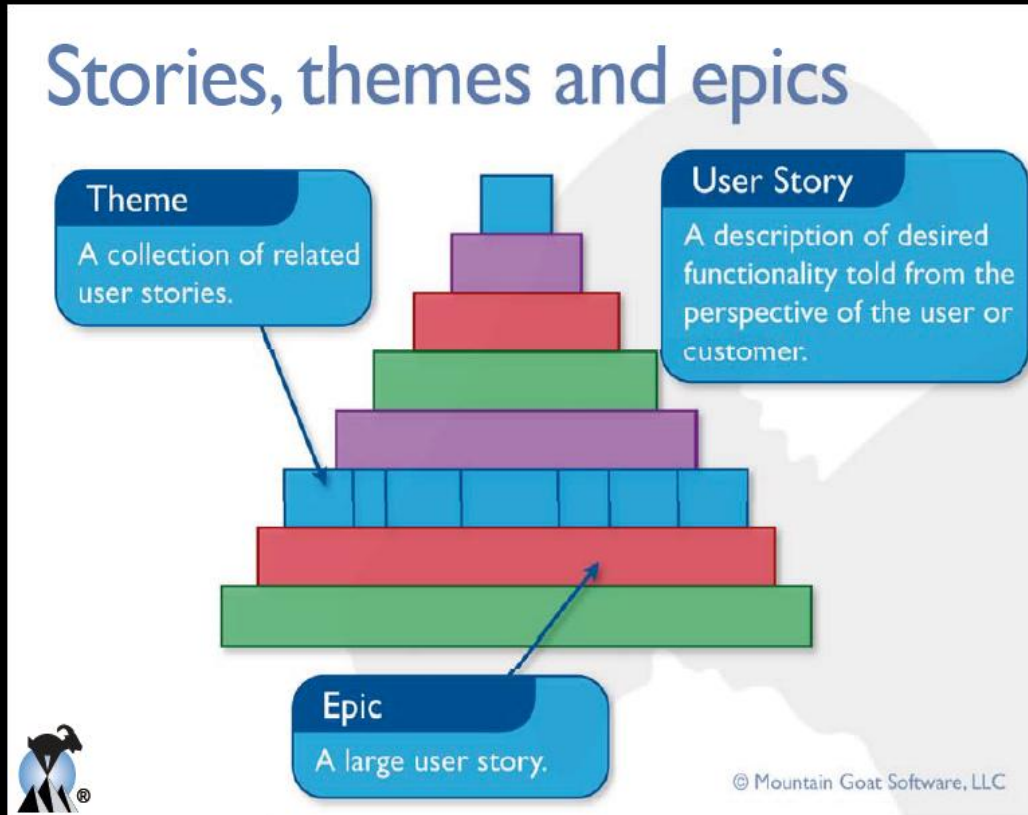
<http://xp123.com/xplor/xp0308/index.shtml>



Algo más sobre las User Stories...

- No son especificaciones detalladas de requerimientos (como los casos de uso)
- Son expresiones de intención, “es necesario que haga algo como esto...”
- No están detallados al principio del proyecto, elaborados evitando especificaciones anticipadas, demoras en el desarrollo, inventario de requerimientos y una definición limitada de la solución.
- Necesita poco o nulo mantenimiento y puede descartarse después de la implementación.
- Junto con el código, sirven de entrada a la documentación que se desarrolla incrementalmente después.

Diferentes niveles de abstracción



Idea

Problema

Necesidad

Cambios en el Negocio

Ideas / Cambios en
el Software

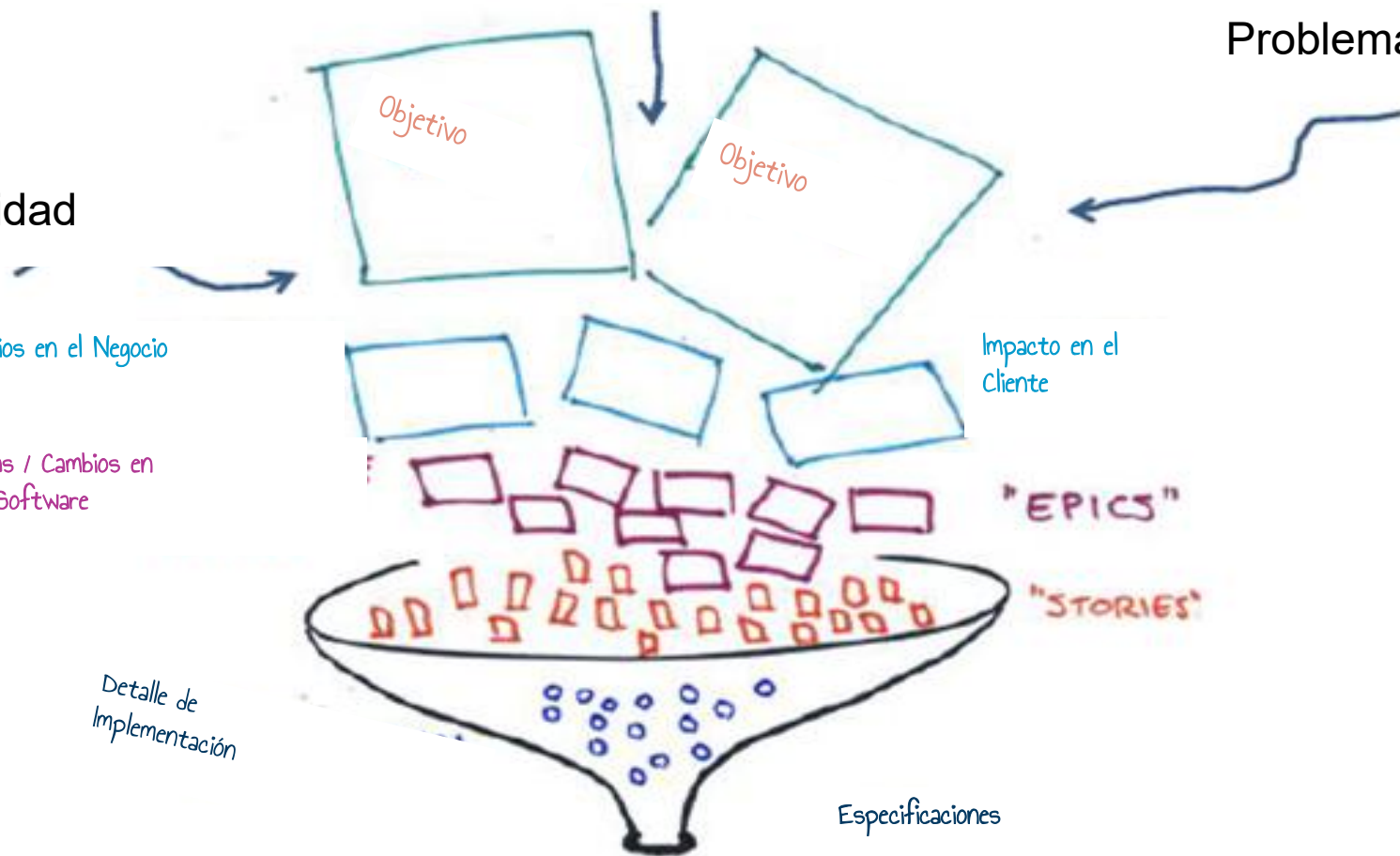
Detalle de
Implementación

Impacto en el
Cliente

"EPICS"

"STORIES"

Especificaciones



Qué no son las user stories.....



User Stories:
Algo “Huele
mal”...

¿Qué huele mal
en las siguientes
historias?



¿Qué huele mal en las siguientes historias?

- El usuario puede ejecutar el sistema en Windows XP y Linux.
- Todos los gráficos se harán con una librería de terceros.
- El usuario podrá deshacer hasta 50 comandos.
- El software deberá entregarse el 30 de Julio.
- El software deberá estar desarrollado en JAVA

¿Qué huele mal en las siguientes historias?

- El usuario podrá seleccionar su país desde una lista desplegable.
- El sistema utilizará Log4J para registrar todos los mensajes de error en un archivo.
- El usuario podrá exportar datos a XML.
- Al usuario se le pedirá guardar su trabajo si no lo hizo luego de 15 minutos.

Y acá que “huele mal”?

- User Story:
 - Como usuario, yo necesito almacenar el ID code con 10-14 dígitos, 4 dígitos para el PIN (el cual es opcional) y muchos RFID tags en mi profile, así yo puedo tener muchos log-ins.
- Criterio de Aceptación
 - AC1: Manualmente agregar el Id de 10-14 dígitos al profile
 - AC2: Manualmente agregar 4 digit PIN al profile
 - AC3: ver el PIN en pagina de profile
 - AC4: Manualmente agregar 3 RFID tag IDS a mi profile
 - AC5: Ver RFID tags en mi página profile.
 - AC6: SI PIN es requerido, poner una bandera y verificar que el PIN se ha proporcionado

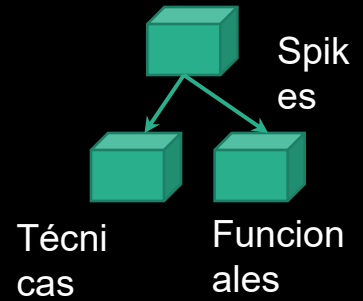


5 minutos

Spikes

- Tipo especial de historia, utilizado para quitar riesgo e incertidumbre de una User Story u otra faceta del proyecto.
- Se clasifican en : técnicas y funcionales.
- Pueden utilizarse para:
 - Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
 - Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
 - Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
 - Frente a riesgos funcionales, donde no está claro como el sistema debe resolverla interacción con el usuario para alcanzar el beneficio esperado.

Spikes



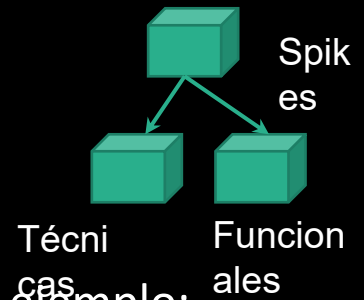
Técnicas

- Utilizadas para investigar enfoques técnicos en el dominio de la solución.
 - Evaluar performance potencial
 - Decisión hacer o comprar
 - Evaluar la implementación de cierta tecnología.
- Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.

Funcionales

- Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema.
- Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Spikes



- Algunas User Stories requieren de ambos tipos de spikes. Por ejemplo:
 - Como un cliente, quiero ver mi uso diario de energía en un histograma, para poder comprender rápidamente mi consumo de energía pasado, presente y proyectado.
- En este caso un equipo puede crear dos spikes:
 - Spike Técnico:
 - Investigar cuanto tiempo requiere actualizar un display de un cliente al uso actual, determinando requerimientos de comunicación, ancho de banda y si los datos se actualizan en formato push o pull.
 - Spike Funcional:
 - Crear un prototipo de histograma en el portal web y obtener la retroalimentación de algunos usuarios respecto del tamaño, el estilo de la presentación y los atributos gráficos.

Lineamientos para Spikes

Estimables, demostrables, y aceptables

La excepción, no la regla

- Toda historia tiene incertidumbre y riesgos.
- El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
- Los spikes deben dejarse para incógnitas mas críticas y grandes.
- Utilizar spikes como última opción.

Implementar la spike en una iteración separada de las historias resultantes

- Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

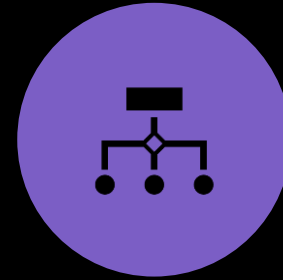
Algunas cosas para dejar en claro



DIFERIR EL ANÁLISIS DETALLADO TAN TARDE COMO SEA POSIBLE, LO QUE ES JUSTO ANTES DE QUE EL TRABAJO COMIENCE.



HASTA ENTONCES, SE CAPTURAN REQUERIMIENTOS EN LA FORMA DE "USER STORIES".



LAS USER STORIES NO SON REQUERIMIENTOS DE SOFTWARE, NO NECESITAN SER DESCRIPCIONES EXHAUSTIVAS DE LA FUNCIONALIDAD DEL SISTEMA.

Tips para que las user stories sean útiles para el equipo



Un paso a la vez
(evitar la palabra “Y”)



Usar palabras claras
en los criterios de
aceptación



No olvides la parte
invisible: la
conversación



Las user stories se
escriben desde la
perspectiva del
usuario



No forzar todo para
escribirlo como user
stories

Material Bibliográfico de Referencia

- Libro:
 - Cohn, Mike - USER STORIES APPLIED – Editorial Addison Wesley 2004- Capítulos 1, 2 y 6
- Papers
 - Dean Leffingwell and Pete Behrens – A user story primer (2009)
- Link
 - <http://www.mountangoatsoftware.com/>